

Lab2: 神经网络加速器计算核心设计

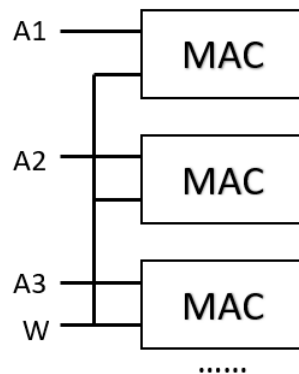
一、作业任务

本次作业要求参考“加速器”、“映射”部分的课程内容，使用 Verilog/SystemVerilog 语言设计实现 CNN 加速器的核心加速模块——计算阵列。

二、参考实现代码

为了方便同学们快速上手本次作业，文档中提供了参考设计代码。包含 `pe_array.sv` 与 `tb_pe_array.sv` 两个文件。前者为阵列设计文件，后者为测试文件。

参考代码的阵列设计，是一组长度为 10 的独立乘累加单元，他们各自进行自己的运算，其整体连接结构如下所示：



每次传输进来的权重 (Weight)，通过广播的方式给到所有的乘累加单元。而每一个乘累加单元，都会收到自己的激活值 (activation)。每一个 MAC (multiply accumulate) 都由一个乘法器，一个加法器及一个长度为 32 的累加寄存器组成。

本次的任务数据也包含在文档中，其运算来自第一次作业所用网络的第二层卷积。其权重为输入 6 通道，输出 16 通道，核尺寸为 5×5 的卷积，其可以被展开成为 150×16 的矩阵。输入激活值为 14×14 的 6 通道数据，可以通过 `img2col` 操作展开成为 100×150 的矩阵。相关展开细节可以参考 kernel computation 相关课程有关 GEMM 的知识。其最终会输出 100×6 的矩阵，可以转换为 10×10 ，通道为 6 的尺寸的输出激活值。

本次作业的 scaling factor 固定为 2^{-8} ，因此在实现时只需用向右算术右移八位即可实现。具体可见参考代码的实现方式。

本次作业只需要进行计算核心的设计，因此对任务的切分及数据搬运操作都在 testbench 中用类似 C 语言的语法实现即可，相关实现方式可以参考给的测试代码。可以看到，任务数据的切分与 mapping 控制，是在 testbench 中模拟的，参考代码的切分方式为：

```

for(int m=0; m<16; m=m+1) {
    for(int j=0; j<100/10; j=j+1) {
        for(int i=0 ; i<150 ; i=i+1) {
            mvin(W[i][m]);
            spatial for(int n=0; n<10 ; n=n+1) {
                mvin(IA[10j+n][i]);
                OA[10j+n][m] += IA[10j+n][i]* W[i][m];
            }
        }
        spatial for(int n=0; n<10 ; n=n+1) {
            mvout(OA[10j+n][m]);
        }
    }
}

```

可以看到是 output stationary 的设计。在这样的 mapping 方式及阵列设计下，假设时钟为 100MHz，通过仿真及其他计算方式，可以得到参考代码的设计参数为：

算力	10*100M=1Gops
IA 带宽需求	8b*10*100M=8000Mbps
W 带宽需求	8b*100M=800Mbps
OA 带宽需求	8b*10*100M=8000Mbps
IA 片上 buffer 容量	8b*10=80bit
W 片上 buffer 容量	8b*1=8bit
OA 片上 buffer 容量	8b*10=80bit
IA 片外访问总 bit 数	16*100*150*8bit
OA 片外写回总 bit 数	16*100*8bit
W 片外访问总 bit 数	16*10*150*8bit

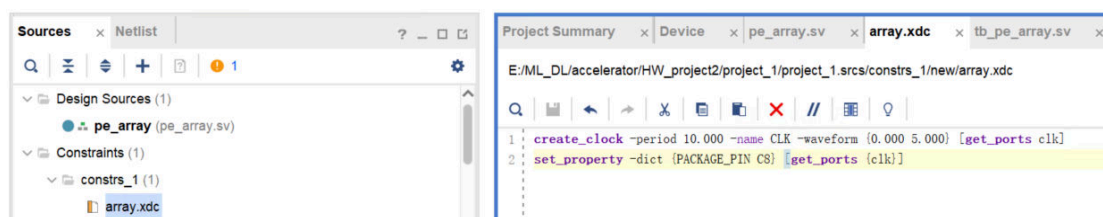
三、有关 SystemVerilog 及仿真工具使用

SystemVerilog 兼容所有 Verilog 的语法，因此同学按照 verilog 的编写习惯完全没有问题，其文件以 .sv 为结尾。与 Verilog 不同的是，其支持高维数组变量作为模块与模块之间的端口连接，因此设计更加方便。

仿真和综合工具可以采用因材施教所讲述的 EDA 工具，也可以用数逻实验所使用的 Vivado 软件。Vivado 推荐采用大于 2019.2 的版本，对 SystemVerilog 语法兼容性更好（否则仿真可能出错）。相关下载链接将在课程群中发布。在 Vivado 中，开发板请选择：

Product family:	Zynq-7000
Project part:	xc7z035ffg676-2

请在约束文件中添加时钟约束代码：



本次实验只需获得综合后的资源消耗及时序参数即可（即 Design Compiler 的综合结果，或 vivado synthesis 的结果）。Run synthesis 之后，点击 report utilization，可以得到参考代码的资源消耗：

Tcl Console

Messages

Log

Reports

Design Runs

Timing

Utilization

Q

≡

⌵

Summary

Hierarchy

Summary

▼ Slice Logic

▼ Slice LUTs (1%)

LUT as Logic (1%)

Resource	Utilization	Available	Utilization %
LUT	2165	171900	1.26
FF	492	343800	0.14
IO	180	250	72.00

点击 report timing summary，可以得到参考代码的时序信息：

Tcl Console

Messages

Log

Reports

Design Runs

Timing

Utilization

Q

≡

⌕

↺

📁

●

Design Timing Summary

General Information

Timer Settings

Design Timing Summary

Clock Summary (1)

Check Timing (99)

Intra-Clock Paths

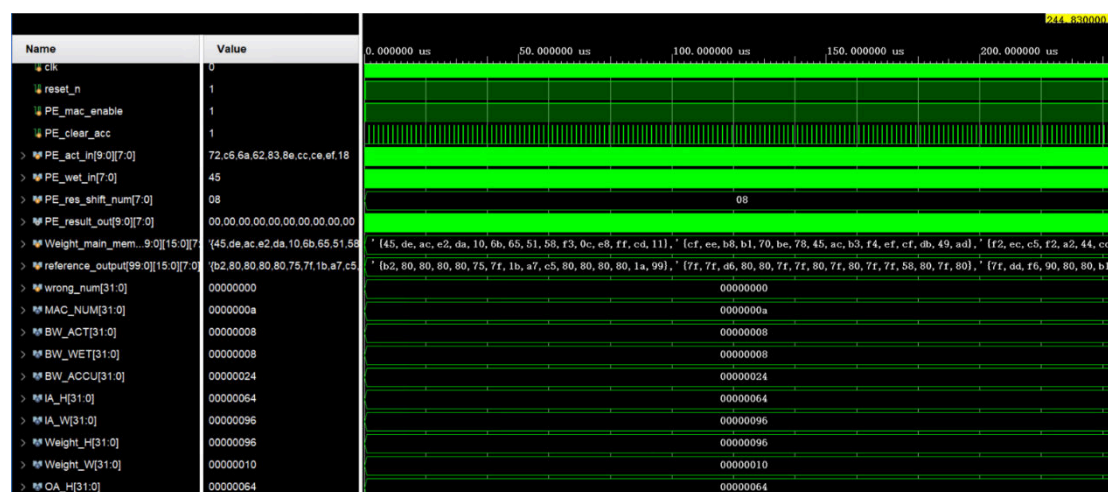
Inter-Clock Paths

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 6.086 ns	Worst Hold Slack (WHS): 0.100 ns	Worst Pulse Width Slack (WPWS): 4.650 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 200	Total Number of Endpoints: 200	Total Number of Endpoints: 251

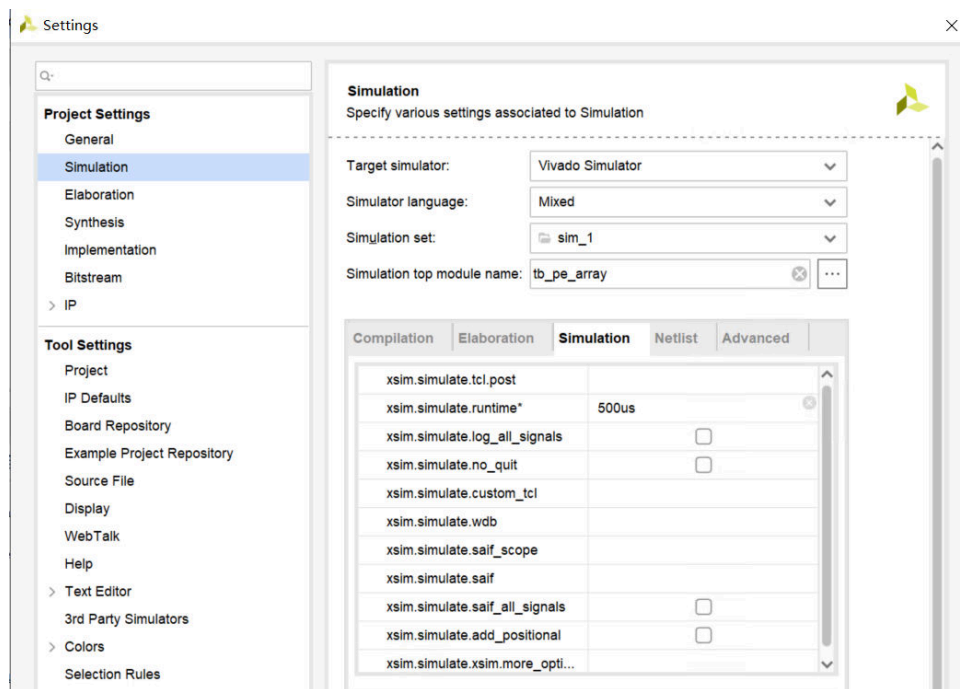
All user specified timing constraints are met.

可见在 100M 时钟下，有 6.086ns 的建立时间裕量，其能用更快的时钟跑。

在 Vivado 的行为级仿真中，该设计在 100M 时钟下，完成任务用时 245.83us。而计算量/算力=100*150*16/1Gops=240us，可见阵列利用率为 240/245.83=97.6%，阵列并不是每时每刻都在计算。



仿真时请右键选择 simulation settings, 可以在其中调整 simulation time 如下所示：



输出如有错误，将会在 vivado 软件下方的 tcl console 中打印，如没有错误，将会输出如下信息：

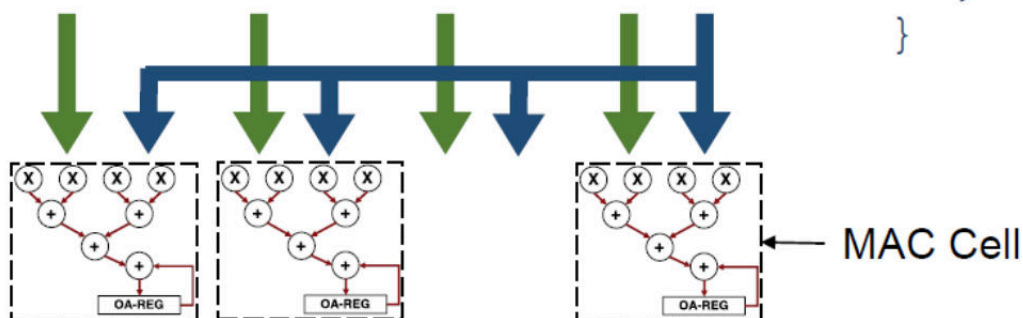
```
# run 500us
wrong num: 0
$finish called at time : 244830 ns
```

这些输出设置在 tb_pe_array. sv 中可以自行阅读学习。

四、基础作业任务

1、乘累加树型阵列设计与实现

乘累加树是 NVDLA 等系统的阵列实现方式，其实现方案如下所示：

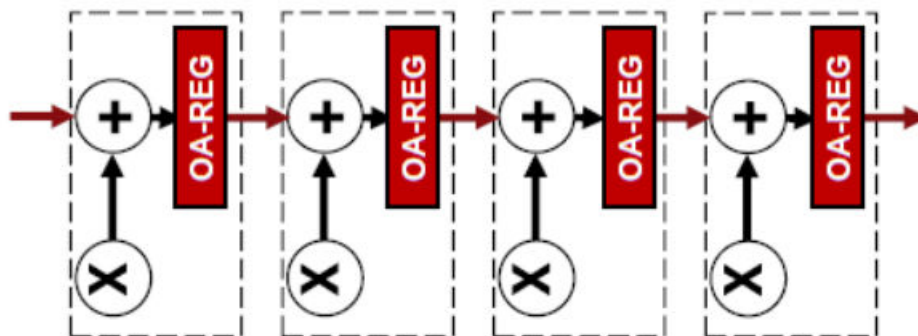


也可参考第七讲 accelerator 相关的知识内容。其特点是在乘法器后面跟了阶数为 $\log N$ 的加法树单元。相邻乘法运算必须是为同一个输出目标而计算，与参考代码中乘法输出相互独立不同。

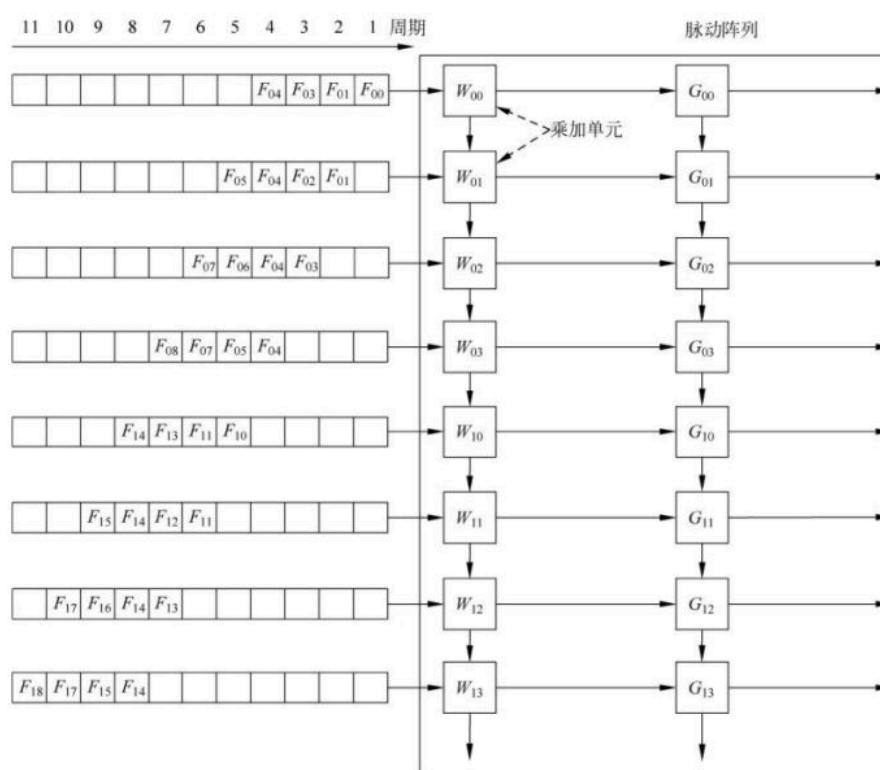
请同学们按照自己的设计，实现一个 $1 \times N$ 的乘累加树阵列，替代参考代码中的阵列实现。同学可自行更改仿真文件，以配合自己的阵列设计。

2、脉动阵列型设计与实现。

脉动阵列是 TPU 及 Eyeriss 等芯片的阵列实现方式，其设计如下所示：



可以看到，与乘累加树直接将乘法结果加和不同，其乘法输出数据是以脉动的方式进行传播的。特别需要注意的是，这意味着右侧的乘法结果需比相邻左侧晚出来一个周期，因此需要改变其数据输入方式（右侧比左侧晚输入一个周期）：



具体可参考第七讲 accelerator 课程的 P50-53 页。请按照自己的设计重新编写 pe array 及 testbench 代码，实现 1*N 的阵列设计。

3、请将上述任意一种阵列扩展为 $M \times N$ ，实现设计。

4、统计和报告你设计的算力、时钟、资源消耗、带宽、片上 buffer 容量，访存 bit 数等数据。

五、思考题

- 1、用非 GEMM 的方式算卷积（比如 Winograd 卷积，或者直接算卷积），该如何控制数据向阵列的传输？
- 2、在你之前设计的基础上，如果有更大的片上 buffer（或者有一个新的 shared buffer），你会怎么重新进行 mapping 的设计？会有什么样的好处？（提示：40nm 工艺下，外部访存的功耗是内部访存的 3~6 倍以上）可以进行一个 case study。
- 3、在你之前设计的基础上，如果实际不能满足你的带宽需求，比如参考代码的 OA 带宽需求是 8000Mbps，但实际只能做到 800Mbps，那该如何重新进行设计？可以进行一个 case study。
- 4、在你之前设计的基础上，如果阵列的尺寸并不是新任务中输入矩阵尺寸的因数，例如在参考代码中，阵列尺寸是 1×10 ，来了个新任务输入激活值矩阵尺寸是 102×128 ，权重矩阵尺寸是 128×18 。那么你该如何进行 mapping？或者如何进行更好的阵列设计？可以进行一个 case study。
- 5、在你之前设计的基础上，试着去搜索一下更好的 mapping 方案。（延时更低、带宽需求更低、访问 bit 数更低等）
- 6、试着优化你的设计，使其能够跑到更高的时钟/更短的任务时间。

以上任务，可以以进行一些数学上的探索为主，也可以选择性的用 Verilog 进行设计验证，最好能结合事例分析。

六、文件附录说明

- 1、pe_array.sv 参考代码中的阵列设计文件
- 2、tb_pe_array.sv 参考代码中的阵列测试文件
- 3、input_act_dec, input_act_bin 分别是输入数据的十进制与二进制表示文件，长 100×150 ，先排列第二维，再排列第一维。
- 4、weight_dec, weight_bin 分别是权重矩阵的十进制与二进制表示文件，长 150×16 ，先排列第二维，再排列第一维。
- 5、reference_output_dec, reference_output_bin，分别是正确输出的十进制与二进制表示文件，长 100×16 ，先排列第二维，再排列第一维
- 6、reference_un, 是乘以 scaling factor (shift num) 之前的输出，长 100×16 ，用以帮助 debug。

Vivado 下载链接：<https://cloud.tsinghua.edu.cn/d/25a3ec9537b544e4b3cc/>

请同学们如遇到问题，积极与助教联系！（包括但不限于因为期中考试，来不及做大作业，想要延后作业汇报时间等等）